

UPS ORION

Как спроектировать ИС, которая меняет маршрут доставки

РЕАЛЬНЫЙ КЕЙС · ВНЕДРЕНИЕ ЗАВЕРШЕНО В 2016 ГОДУ

БЫЛО

Решение по опыту

ИЗМЕНЕНИЕ

Оптимизация + данные

СТАЛО

Порядок остановок

Работу выполнила команда «Байтенки»

Один процесс вместо обзора всей компании

UPS

Логистика и доставка

Разработчик ORION —
сама UPS

Объект анализа

Дневной маршрут
одного водителя

Доставки + заборы

Вне границ

Межгород и сортировка

Оптимизация всей сети
и поздние версии

Задания дня —→ **Последовательность** —→ **Выполнение**

Рост сложности делает опыт недостаточным

Меньше плотность

Больше расстояние
между остановками

Больше условий

Ожидания клиентов
и требования сервиса

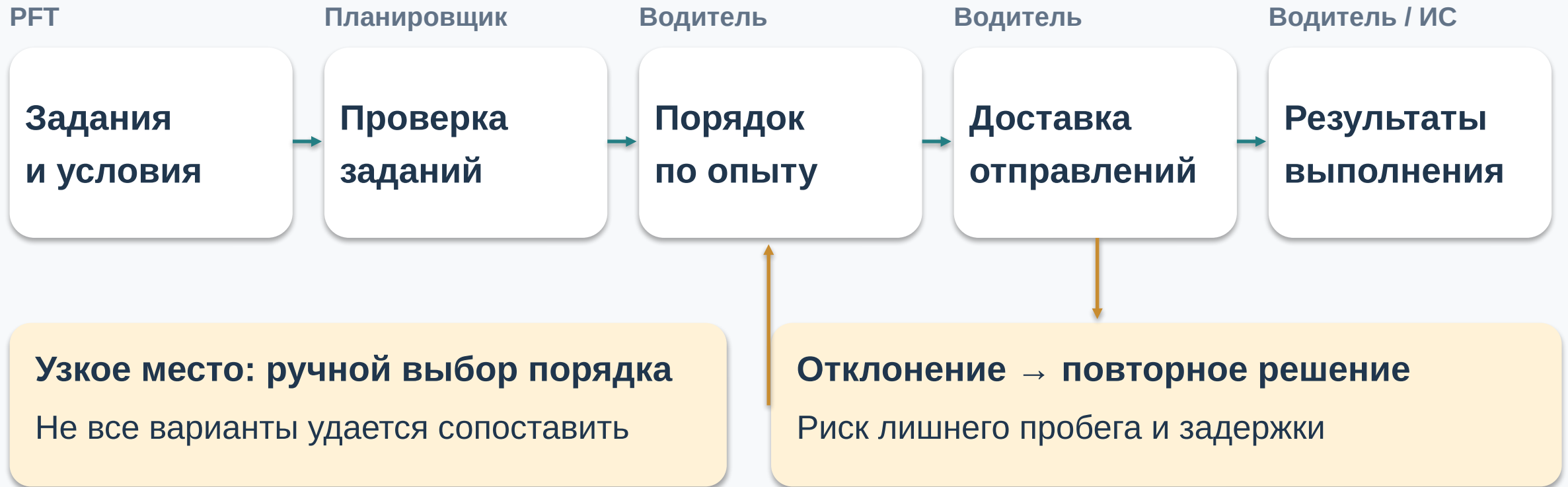
Сложнее выбор

Снизить пробег,
сохранив обслуживание

Задача ИС — улучшить решение, а не просто оцифровать работу

AS-IS: порядок уточняет человек

ТЕ ЖЕ ВХОДЫ И ГРАНИЦЫ ПРОЦЕССА



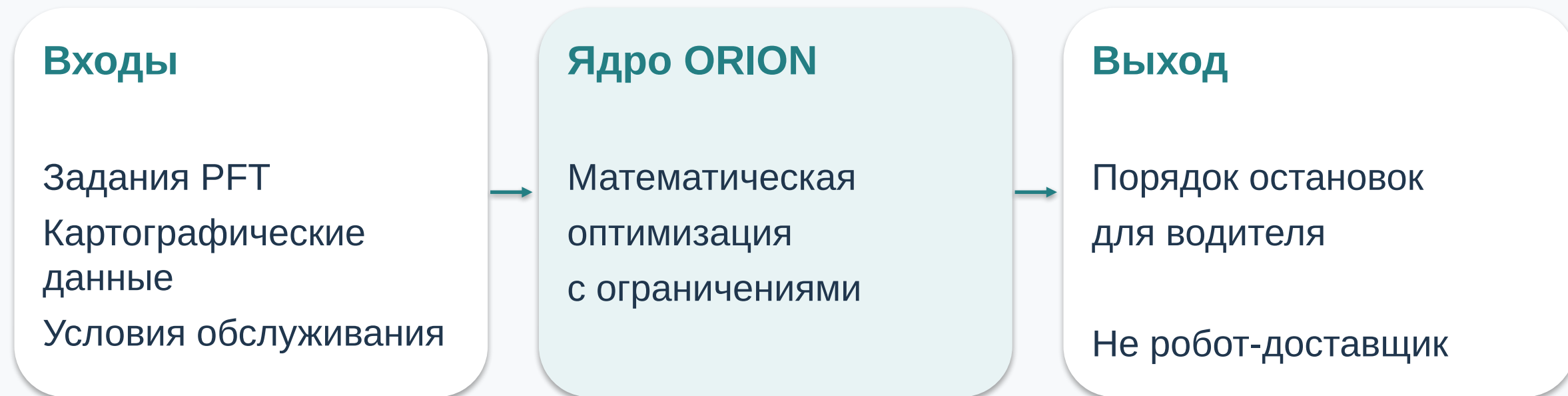
До ORION уже была PFT. Это не переход «с бумаги в компьютер».

Каждой проблеме — проверяемое требование

ПРОБЛЕМА	ТРЕБОВАНИЕ	ЧТО ПРОВЕРЯЕМ
Ручное сравнение	Оптимизация с ограничениями	Пробег при том же задании
Неточные карты	Контроль качества данных	Пригодность входов
Непринятие маршрута	Учет опыта + обучение	Выполнимость и принятие

Нет данных о частоте ошибок — нет вымышленных процентов.

ORION автоматизирует выбор последовательности

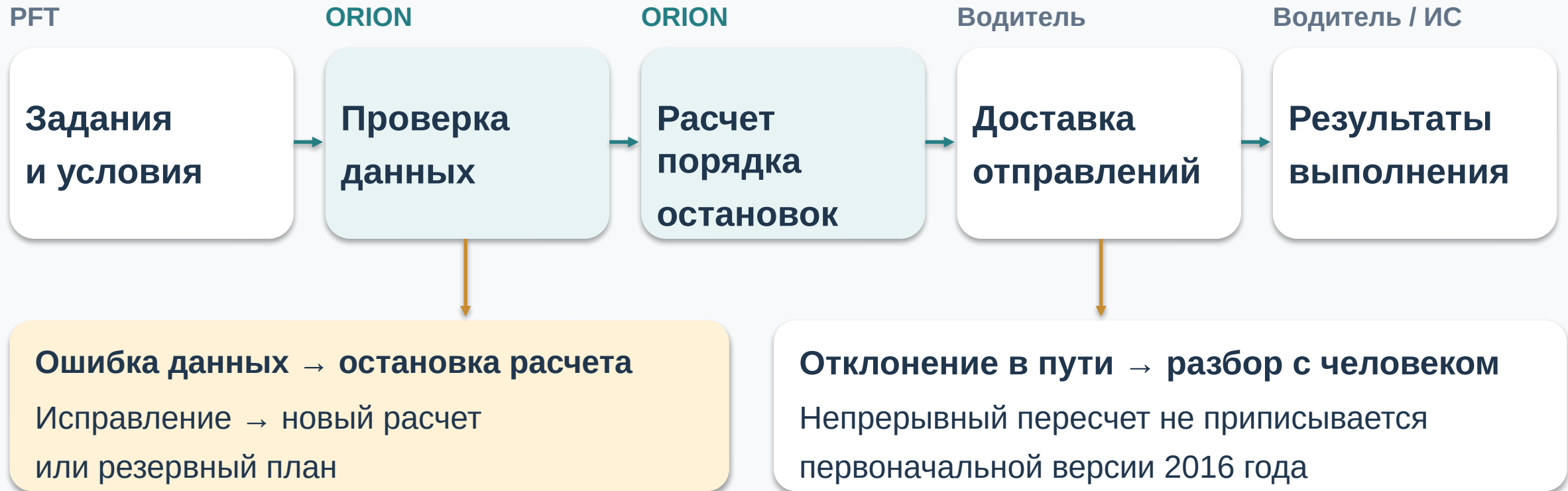


> 200 000

вариантов маршрута анализируются за секунды

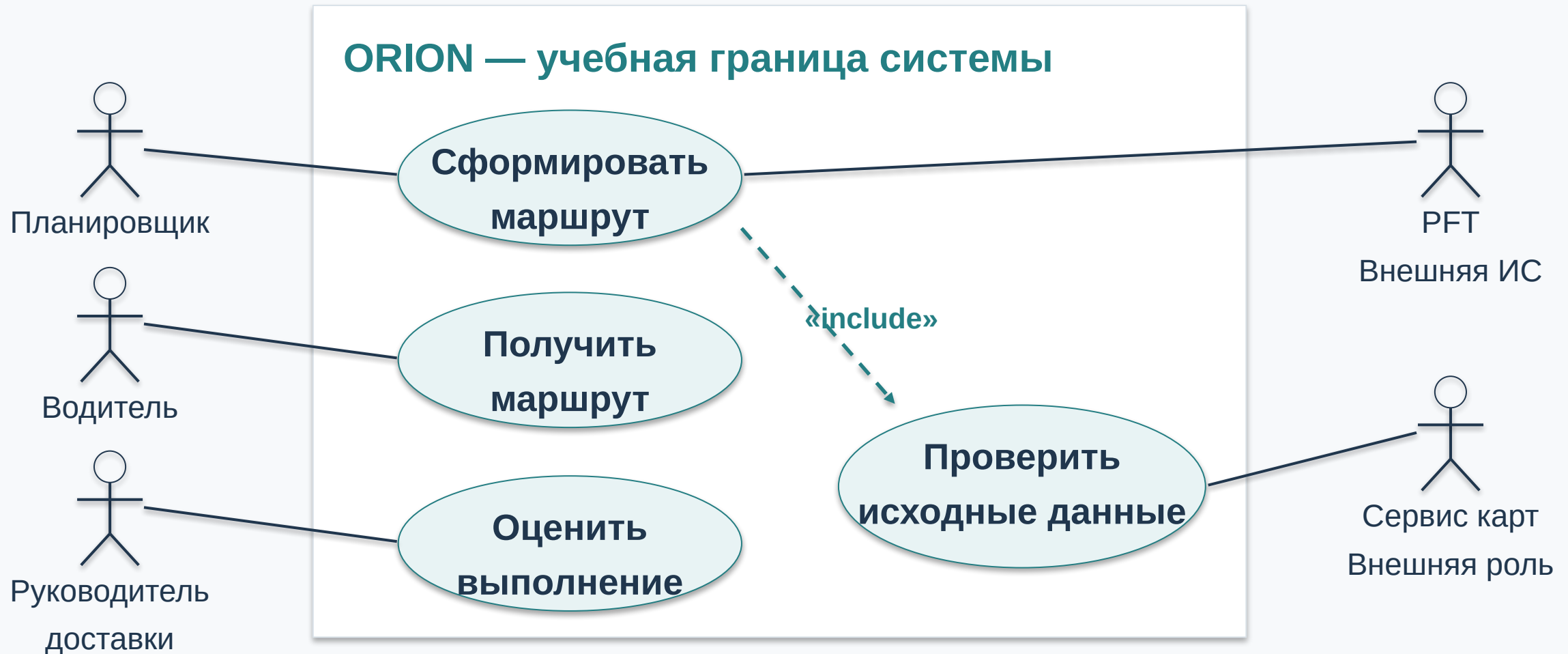
ТО-BE: система считает, человек выполняет

ТЕ ЖЕ ВХОДЫ И ГРАНИЦЫ ПРОЦЕССА



Автоматизировано решение о последовательности. Доставку выполняет человек.

Варианты использования: цели, не код



Архитектура: слои известны только логически

РЕКОНСТРУКЦИЯ · МНОГОСЛОЙНАЯ КЛИЕНТ-СЕРВЕРНАЯ МОДЕЛЬ

Клиенты: планировщик · водитель · руководитель

PFT
Подтверждено [1]

Прикладной слой ORION

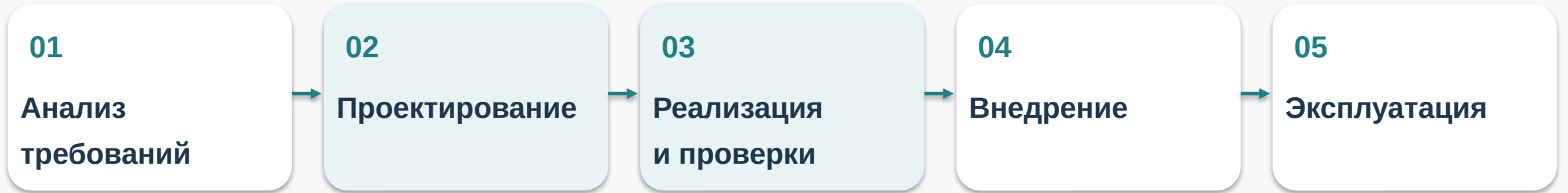
Проверка входов → собственное ядро UPS → выдача маршрута

Карты
Подтверждено [1]

Данные: задания · ограничения · версии маршрутов

Облако? СУБД? Микросервисы? В найденных источниках не установлены.

ЖЦ: итерации разработки, внедрение по частям



←
Обратная связь полевых тестов

Перед площадкой

Данные и персонал готовы

После площадки

Критерии приемки выполнены

График проекта: что подтверждено источниками

Прототипирование → запуск → распространение решения

Гант и вехи по фактическим сведениям: точность до года



Годы. Положение маркера внутри года условно; месяцы не установлены.

Результат: меньше пробег и стоимость обслуживания

БЫЛО

PFT + опыт водителя

Общий пробег и затраты
в источнике не раскрыты



СТАЛО

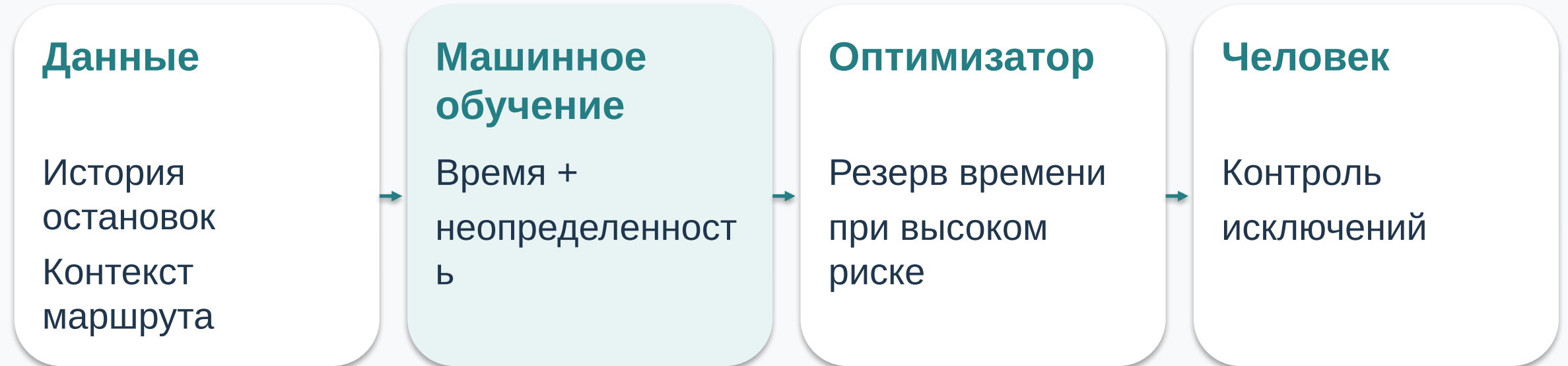
PFT + ORION

–100 млн миль / год
\$300–400 млн / год экономии

Оценки UPS в интервью 2017 года [1]. Эффект ORION отдельно от PFT. Процент снижения не вычисляется.

Нет абсолютной базы → нельзя честно показать процент снижения или ROI.

Развитие: прогноз времени остановки и его риска



Проверка в пилоте: меньше опозданий без ухудшения пробега и безопасности

При недоверии к модели → базовый расчет. Минимизация персональных данных.

Успех ИС = решение + данные + изменение работы

Решение

Выбор порядка
остановок

Данные

Точные карты
и ограничения

Люди

Испытания
и принятие

Опорные источники

[\[1\] Jack Levis, UPS — интервью ODBMS, 2017](#)

[\[4\] INFORMS / ORMS Today — «ORION: результат для UPS», 2016](#)

[4] Архив: 16.05.2025. [5] UPS — пресс-релиз о запуске, 12.04.2016 (GlobeNewswire).

Вопросы другим мини-группам

- 1 Какое звено AS-IS ограничивало качество решения?
- 2 Почему ошибка карты может обесценить хороший алгоритм?
- 3 Почему нельзя назвать архитектуру микросервисной по этой схеме?
- 4 Что изменить при росте нагрузки в десять раз?
- 5 Какой пилот докажет пользу ML, а не только точность прогноза?

Главный вывод: проектировать нужно не алгоритм отдельно, а весь процесс принятия решения.